

Lab3 Report: MaskGIT for Image Inpainting

張苙烜/ 資科工碩 313551079

1 Introduction

這次作業的目標是使用 MaskGIT 完成 Image Inpainting。分為兩個部分，首先先用 VQVAE encode image 到對應的 latent space z ，轉換成 token 之後再實作出 Bi-directional Transformer 去預測正確的原始值，透過最小化 cross entropy loss，重建圖像中被遮蔽的區域。並且嘗試使用不同的 Mask Scheduling Function (Linear、Cosine、Square) 進行多輪修復，每一輪根據模型信心分數更新 token，逐步填補被遮蔽區域，並儲存不同輪次的視覺化結果供觀察比較。最終以 FID 作為圖像生成品質評估指標，並觀察不同設定下的結果差異。

2 Implementation Overview

2.1 The details of your model (Multi-Head Self-Attention)

- 輸入是 (B,N,C)，(Batch size,tokens number,Channel)
- Q,K,V 各自分成 16 組 attention heads，且每個 heads 的維度是 $768/16=48$

在 forward 階段，將輸出 reshape 成 (B, N, 3, num_heads(16), head_dim(48))，然後再交換成 (3, B, num_heads(16), N, head_dim(48))，最後可以得到每個 head 各自的 QKV(B, 16, N, 48)，對每個 head 計算 self-attention score，公式如下：

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

最後所有的 head 的輸出被 concatenate 接回原始維度 (B, N, C)，再透過一層 Linear (self.proj) 將結果映射回輸入維度。

```
1 class MultiHeadAttention(nn.Module):
2     def __init__(self, dim=768, num_heads=16, attn_drop=0.1):
3         super(MultiHeadAttention, self).__init__()
4
5         self.num_heads = num_heads
6         self.head_dim = dim // num_heads # 768 // 16 = 48 head_dim means d_k and d_v
7         self.scale = self.head_dim ** -0.5 # 1 / sqrt(d_k)
8         self.qkv = nn.Linear(dim, dim * 3) # q, k, v
9         self.proj = nn.Linear(dim, dim)
10        self.attn_drop = nn.Dropout(attn_drop)
11
12    def forward(self, x):
13        ''' Hint: input x tensor shape is (batch_size, num_image_tokens, dim),
14            because the bidirectional transformer first will embed each token to dim dimension,
15            and then pass to n_layers of encoders consist of Multi-Head Attention and MLP.
16            # of head set 16
17            Total d_k , d_v set to 768
18            d_k , d_v for one head will be 768//16.
19        '''
20        B, N, C = x.shape
21        # use permute to change B N 3 16 48 -> 3 B 16 N 48
22        qkv = self.qkv(x).reshape(B, N, 3, self.num_heads, self.head_dim).permute(2, 0, 3, 1, 4)
23        q, k, v = qkv[0], qkv[1], qkv[2]
24
```

```

25     attn = (q @ k.transpose(-2, -1)) * self.scale # attention() = softmax((Q * K^T) / sqrt(
        d_k))
26     attn = attn.softmax(dim=-1)
27     attn = self.attn_drop(attn)
28
29     x = (attn @ v).transpose(1, 2).reshape(B, N, C)
30     x = self.proj(x)
31     return x

```

Listing 1: Multi-Head Attention in PyTorch

2.2 The details of your stage2 training (MVTM, forward, loss)

2.2.1 MVTM & forward

透過預先訓練好的 VQVAE 將輸入 x encode 到 z ，也就是下面的 `encode_to_z(self, x)`。之後隨機遮蔽這些 token 中的一部分，替換成預設的 `mask_token_id`，被遮蔽的位置由 `mask scheduling function(cosine, linear, square)` 決定。再將經過遮蔽的 `z_masked` 作為輸入，餵入 Bidirectional Transformer 中，預測每個 token 的 logits (B, N, K 為可能的 codebook)，回傳 `z_indices` 與 logits 作為後續比對。

```

1  ##TODO2 step1-1: input x fed to vqgan encoder to get the latent and zq
2      @torch.no_grad()
3      def encode_to_z(self, x):
4          codebook_mapping, codebook_indices, _ = self.vqgan.encode(x)
5          return codebook_indices.reshape(codebook_mapping.shape[0], -1)
6
7  ##TODO2 step1-2:
8      def gamma_func(self, mode="cosine"):
9          """Generates a mask rate by scheduling mask functions R.
10
11             Given a ratio in [0, 1), we generate a masking ratio from (0, 1].
12             During training, the input ratio is uniformly sampled;
13             during inference, the input ratio is based on the step number divided by the
14             total iteration number: t/T.
15             Based on experieiments, we find that masking more in training helps.
16
17             ratio: The uniformly sampled ratio [0, 1) as input.
18             Returns: The mask rate (float).
19
20             """
21             if mode == "linear":
22                 return lambda ratio: 1 - ratio
23             elif mode == "cosine":
24                 return lambda ratio: np.cos(ratio * np.pi / 2)
25             elif mode == "square":
26                 return lambda ratio: 1 - ratio ** 2
27             else:
28                 raise NotImplementedError
29
30  ##TODO2 step1-3:
31      def forward(self, x):
32          # step1: encode the input image to latent space z
33          z_indices = self.encode_to_z(x)
34
35          B, N = z_indices.shape
36          device = z_indices.device
37          # step2: initialize the mask
38          r = torch.rand(B)
39          num_mask = (r * N).long()

```

```

40     mask = torch.zeros_like(z_indices, dtype=torch.bool) # 0: not masked, 1:
        masked initial don't mask any token
41     for i in range(B):
42         perm = torch.randperm(N)
43         mask[i, perm[:num_mask]] = True
44
45     # step3: replace the mask token with the mask token id
46     z_masked = z_indices.clone() # copy the z_indices
47     z_masked[mask] = self.mask_token_id # mask token id
48
49     # step4: feed the masked z to transformer
50     logits = self.transformer(z_masked) # fget the probability of tokens
51
52     return logits, z_indices

```

2.2.2 loss

這部分開始訓練，藉由上面寫好的 model 去預測出 logits(每個位置預測它屬於哪個 token 的分數) 和 ground truth `z_indices` (他真正對應的 token index) 後，透過 cross entropy 計算 loss，最大化正確 token 的機率。其公式如下：

$$\mathcal{L} = \text{CrossEntropy}(\text{logits.view}(-1, K), z_{\text{indices.view}}(-1)) \quad (1)$$

```

1     def train_one_epoch(self, train_loader, epoch, args):
2         losses = []
3         self.model.train()
4         for i, data in enumerate(tqdm(train_loader)):
5             data = data.to(args.device)
6             logits, z_indices = self.model(data)
7             loss = F.cross_entropy(logits.view(-1, logits.size(-1)), z_indices.view
                (-1))
8             loss.backward()
9             losses.append(loss.item())
10
11         self.optim.step()
12         self.optim.zero_grad()
13
14         epoch_loss = np.mean(losses)
15         self.scheduler.step()
16         self.writer.add_scalar("Loss/train", epoch_loss, epoch)
17         if epoch % 10 == 0:
18             train_transformer.save_loss_plot(epoch)
19         return epoch_loss

```

2.3 The details of your inference for inpainting task (iterative decoding)

iterative decoding 一步步地將被遮住的 token 位置逐漸填回最有可能的 token，直到全部完成。透過 transformer 的 forward pass 預測出最大機率的位置作為目前預測的 token，接下來計算每個位置的 confidence 判斷哪些可以填補上，哪些需要繼續遮擋預測。越後期會更新更多 token，並且根據不同的 gamma 控制要保留多少 token (不同 mask_scheduling)。採用 Gumbel noise 作為隨機選擇的干擾，將 $(1-\text{ratio}) * \text{self.choice_temperature}$ 在不同輪更新 temperature 大小，越往後隨機性越低。

$$\text{confidence} = z_{\text{indices_predict_prob}} + \text{temperature} * g \quad (2)$$

把那些不是 mask 的地方設成 infinity (不會動)，然後根據信心排序，把信心最低的 token 保留 (下一輪再猜)，信心高的就更新掉用模型預測值填進去，最后回傳新的 `z_indices` 和新的 mask。

```

1  ##TODO3 step1-1: define one iteration decoding
2  @torch.no_grad()
3  def inpainting(self, z_indices, mask_b, step, total_iter):
4      """
5          z_indices:    tensor of shape (B, 256) → 部分位置是 token，部分是 mask token
6          mask_b:      tensor of shape (B, 256) → bool，True 表示這個位置被遮住
7          step:        第幾輪 iterative decoding (從 0 開始)
8          total_iter:   總共 decoding 幾次
9          """
10     B,N = z_indices.shape
11     device = z_indices.device
12     # predict the token probability
13     z_indices_input = z_indices.clone()
14     z_indices_input[mask_b] = self.mask_token_id
15     logits = self.transformer(z_indices_input)
16
17     probs = torch.softmax(logits, dim=-1) # shape: (B, 256, 512)
18
19     #FIND MAX probability for each token value
20     z_indices_predict_prob, z_indices_predict = probs.max(dim=-1)
21
22     # confidence scheduling
23     ratio= (step+1) / total_iter
24     ratio = self.gamma(ratio)
25     #predicted probabilities add temperature annealing gumbel noise as confidence
26     g = - torch.empty_like(z_indices_predict_prob).exponential_().log() # gumbel
       noise(0,1)
27     temperature = self.choice_temperature * (1 - ratio)
28     confidence = z_indices_predict_prob + temperature * g
29
30     #hint: If mask is False, the probability should be set to infinity, so that
       the tokens are not affected by the transformer's prediction
31     #sort the confidence for the rank
32     #define how much the iteration remain predicted tokens by mask scheduling
33     ##At the end of the decoding process, add back the original(non-masked) token
       values
34
35     mask_bc= mask_b.clone()
36     confidence[~mask_bc] = float('inf') # dont change the non-masked token
37
38     mask_counts = mask_bc.sum(dim=1).float() # 每筆的 mask 總數
39     num_remain = (ratio * mask_counts).long() # 每筆應保留更新的數量
40
41     _, rank = torch.topk(confidence,k=confidence.shape[1], dim=-1, largest=False)
42
43     new_mask = torch.zeros_like(mask_bc)
44     for i in range(B):
45         k = num_remain[i].item()
46         if k > 0:
47             keep = rank[i, :k]
48             new_mask[i, keep] = True
49
50     new_z_indices = z_indices.clone()
51     update_mask = ~(new_mask) & mask_bc
52     new_z_indices[update_mask] = z_indices_predict[update_mask]
53
54     return new_z_indices, new_mask

```

3 Discussion

這邊在訓練階段嘗試不同的 gamma type 把 mask scheduling 放進去訓練比較發現有意想不到的結果。確實相較於再訓練階段隨機生成 mask 而言他沒辦法收斂，也很容易 overfitting。驗證了這樣會讓模型太習慣固定樣式的遮罩比例（如：先遮很多、後遮少）導致泛化的能力下降。這邊 linear 會有不錯的表現是因為相較 cosine 和 square 他的遮蔽下降方式較溫和很接近 random 決定。（Table 1顯示比較結果）。

- learning rate: $1e4$ ，50 個 epoch 之後 $\times 0.9$ ，跑 100 epochs
- 當中的 _w 是指 learning rate 有經過 warm up 步驟下的結果

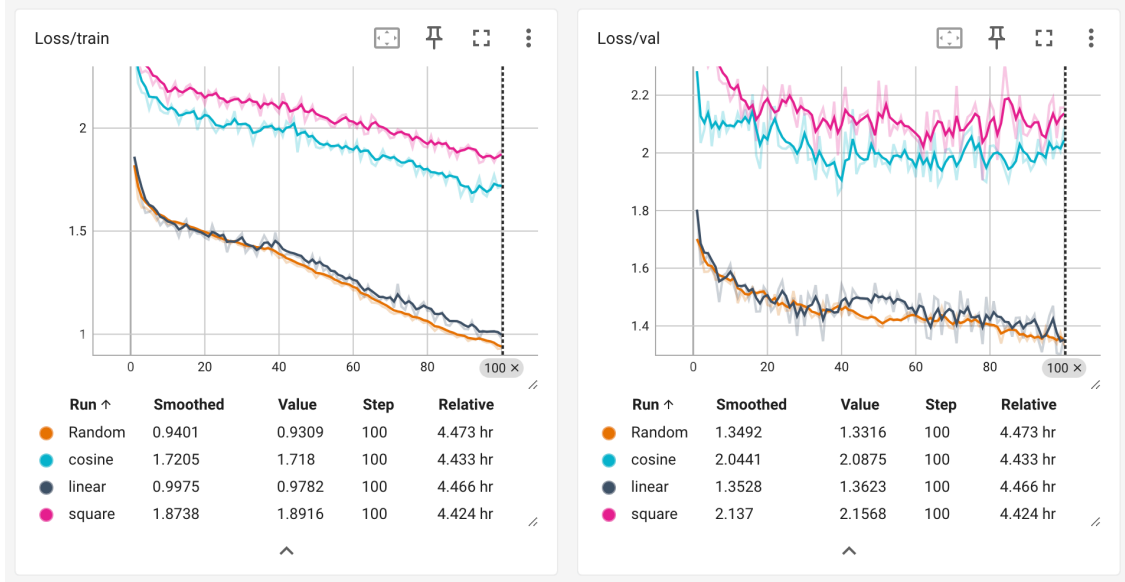


Figure 1: Train/Val loss

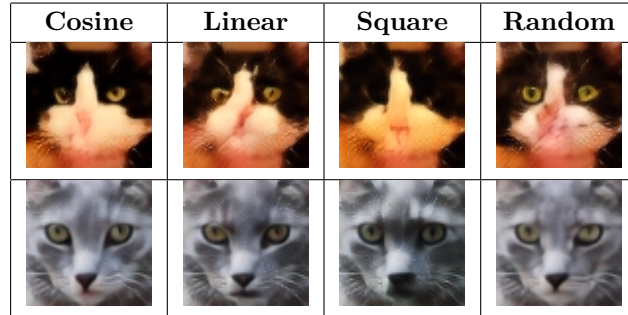


Table 1: Comparison of Different Mask Scheduling Functions in Training Ste

4 Experimental Results

4.1 Different gamma type in inpainting step

從 Table 3 中可以觀察到 linear 呈現平均分布，每輪減少相同比例的遮罩，而 cosine 跟 square 有類似的 scheduling 方式。這邊的實驗只會更動 inpainting 時候的 scheduling 方法，training 時使用隨機的 mask。其對應的 FID 結果如 Table 5 可以看到不同的 scheduling 差別不大，但是 cosine 有最好的 FID 結果。

Predicted image 步驟圖如Table 2所顯示，有正確預測圖片，並且 mask 有逐步相除成功填補。Table 4 有原始圖跟結果圖的比對。Figure 3是 screenshot of results，依序是 cosine、linear、square。超參設定如下：

- epoch 100
- learning rate $1e4$ (每 20 個 epoch learning rate*0.8)
- sweet-spot 8
- total-iter 10

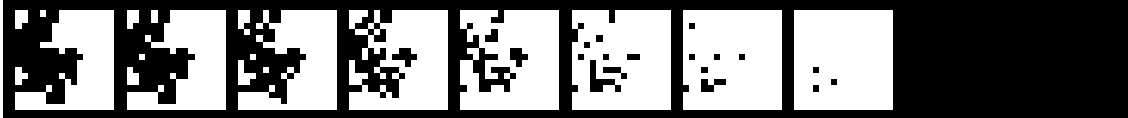
Image								
								
Mask								

Table 2: Predicted image

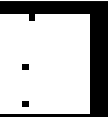
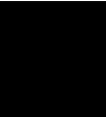
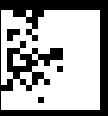
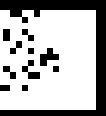
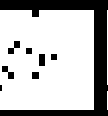
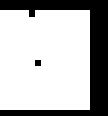
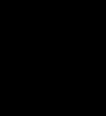
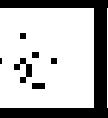
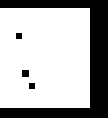
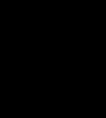
cosine										
linear										
square										

Table 3: Mask scheduling Comparison

4.2 discussion about learning rate setting

從 Figure 1中觀察到大概在 30 個 epoch 之後 val loss 開始沒辦法隨者 train loss 一起下降，這代表訓練結果有點 overfitting 了。在這類型深層的網路架構中，可以嘗試 learning rate 除了 decay 外，也加入 warm up 機制幫助訓練，或是增強 data augmentation 幫助訓練。

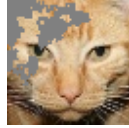
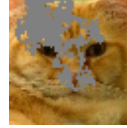
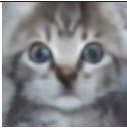
Masked Images					
MaskGIT Inpainting Results					

Table 4: Masked Images v.s MaskGIT Inpainting Results

Mask Scheduling	FID ↓
Cosine	27.36
Linear	27.74
Square	27.65

Table 5: FID scores for different Mask Scheduling Functions.

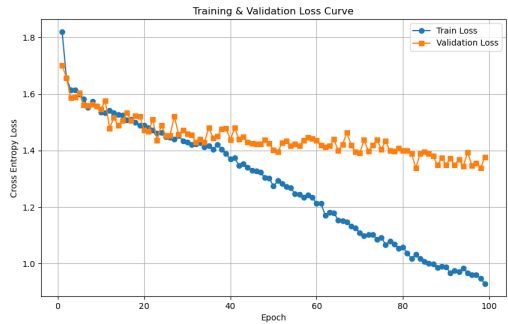


Figure 2: Train and Val Loss Comparison

```

747
100%|██████████| 15/15 [00:01<00:00, 9.26it/s]
100%|██████████| 15/15 [00:00<00:00, 16.96it/s]
FID: 27.355851467719447
> python fid_score_gpu.py --predicted-path ../test_results --device cuda:2

747
100%|██████████| 15/15 [00:01<00:00, 9.12it/s]
100%|██████████| 15/15 [00:00<00:00, 16.30it/s]
FID: 27.743694532548574
> python fid_score_gpu.py --predicted-path ../test_results --device cuda:2

747
100%|██████████| 15/15 [00:01<00:00, 9.08it/s]
100%|██████████| 15/15 [00:00<00:00, 16.37it/s]
FID: 27.64640526568209

```

Figure 3: Screenshot of Results